Сегодня в банке:

- лимиты — в одном сервисе
- антифрод — в другом
- AML — в третьем
- скоринг — в четвертом
- часть правил в АБС
- часть в коде
- часть в BPM
- часть «в голове у Васи»

И каждый раз:

❌ логика размазана
❌ непонятно, кто принял решение
❌ нельзя объяснить “почему отказ”
❌ тяжело менять правила
❌ тяжело тестировать
❌ тяжело трассировать

Но бизнес спрашивает простое:

❓ «Почему операция прошла / не прошла?»

И вот тут начинается ад.

Ключевая мысль FinCompute Hub (далее - Decision Hub)

> **Решение по операции — это не код сервиса.**
> **Это результат вычислений.**

Не “PaymentMS решил”.

А:

> операция прошла, потому что:
>
> - лимит ок
> - антифрод ок
> - AML ок
> - риск-профиль ок

То есть:

> 🔹 решение = сумма независимых проверок

💡 **Аналогия (очень простая)**

FinCompute Hub (лучше называть Decision Hub)

- есть операция → перевод / платёж / выдача кредита
- сеть разбивает решение
- разные модули считают свою часть
- итоговое решение собирается

🎯 **Что такое FinCompute Hub простыми словами**

> **Это не платёжная система.**
> **Это не банк.**
> **Это не ядро.**

Это:

- слой принятия решений

Он **ничего не списывает**,
не переводит деньги,
не владеет счетами.

Он только отвечает на вопрос: “Можно ли это делать?”

📦 Что такое «вычислительный модуль»

Каждый модуль — это независимая логика.

Например:

- **Limit Engine**

вход: клиент, сумма, тип операции
выход: ok / reject + причина

- **Fraud Engine**

вход: клиент, устройство, история
выход: risk_score

- **AML Engine**

вход: получатель, страна, MCC
выход: allowed / blocked

🔹 **Risk Engine**

вход: профиль клиента
выход: risk_level

Каждый модуль:

- независим
- может обновляться отдельно
- может принадлежать разной команде
- может быть заменён

❗ и это ключевая точка.

🧩 **Как сейчас это делается в банке**

Сейчас:

```
PaymentMS
  ├── if limit
  ├── if antifraud
  ├── if aml
  ├── if risk
  └── return result
```

❌ всё в одном месте
❌ сложно менять
❌ сложно тестировать
❌ логика смешана
❌ никто не владеет решением целиком

✅ Как это выглядит в FinCompute Hub

```
PaymentMS
  ↓
DecisionRequest
  ↓
FinCompute Hub
  ├── Limit Module
  ├── Fraud Module
  ├── AML Module
  ├── Risk Module
  ↓
DecisionResult
```

PaymentMS просто спрашивает:

> «Можно или нельзя?»

Он **не знает почему**.

Почему — знает Decision Layer.

🧠 Почему это децентрализация (по-настоящему)

Децентрализация — это не блокчейн.

Децентрализация = отсутствие одного владельца логики.

В FinCompute Hub:

- лимиты может владеть одна команда
- антифрод — другая
- AML — третья
- риск — четвёртая

И они:

- не лезут в код друг друга
- не деплоятся вместе
- не ломают прод друг другу

👉 **нет монолита решений**.

Это и есть децентрализация.

🏦 **Почему это идеально ложится на банковскую философию**

Потому что регулятор всегда спрашивает:

> ❓ “Кто принял решение?”

А ты можешь ответить:

> Решение сформировано из независимых модулей:
> лимиты — служба X
> AML — служба Y
> риск — служба Z

Это **золото для аудита**.

🔍 Почему это полезно лично тебе

Потому что ты сможешь показать команде:

- как правильно отделять бизнес-решения от сервисов
- почему middleware ≠ бизнес-логика
- где должна жить трассировка
- как выглядят best practices TO-BE
- почему ТЗ должно описывать decision flow, а не endpoints

Ты показываешь не “код”.

Ты показываешь **мышление банка будущего**.

🧩 Как это выглядит в виде проекта (sandbox)

Не продукт.
Не MVP.

А **демонстрационная архитектура**:

```
mobile-client (fake)
  ↓
api-gateway
  ↓
payment-service
  ↓
decision-hub
  ├── limit-service
```

```
├── fraud-service
├── aml-service
└── risk-service
```

- логирование
- correlationId
- retry
- idempotency

Ты показываешь коллегам:

> «Вот почему сейчас у нас боль.
> Вот как это должно выглядеть.»

❓ Зачем вообще Decision Hub (эта прослойка)?

Если коротко:

> **Чтобы бизнес мог управлять решениями,**
> **а не кодом.**

И чтобы банк мог **объяснить любое “да / нет”**.

🧠 Сейчас как выглядит реальность (AS-IS)

Возьмём простой кейс:

Клиент делает перевод

Он получает один из ответов:

- ✅ успешно
- ❌ отказано

Бизнес задаёт вопрос:

> ❓ Почему отказ?

И начинается цирк:

- Payment говорит: «у нас всё ок»
- Antifraud говорит: «мы просто дали risk=0.82»
- Лимиты говорят: «мы лимит вернули»
- AML говорит: «мы вообще не участвовали»

❌ нет одного владельца решения
❌ нет объяснения
❌ нет прозрачности
❌ нет трассируемости

👉 решение “растворено” по сервисам.

🔥 Ключевая бизнес-проблема

> **Решение по операции не принадлежит никому.**

А значит:

- его нельзя объяснить клиенту
- его нельзя защитить перед регулятором
- его нельзя анализировать
- его нельзя улучшать

Это **прямая бизнес-боль**, не IT.

💥 Где реальные потери денег

1️⃣ Потери на false positive

Антифрод часто режет:

- нормальных клиентов
- хорошие операции

Каждый такой отказ =

- потерянная комиссия
- потерянное доверие
- потерянный LTV

Но бизнес **не знает**:

> где именно операция “сломалась”

Потому что логика размазана.

2️⃣ Регулятор и комплаенс

ЦБ задаёт вопрос:

> ❓ Почему клиенту отказали?

Ответ сегодня:

> «Ну… там антифрод… потом лимит… потом ещё что-то…»

Это **юридически слабый ответ**.

С Decision Hub ответ становится:

> ❌ отказ по причине AML-правила #AML-102
> значение риска 0.91
> правило утверждено комитетом от 12.03

💰 Это снижение регуляторного риска.

3️⃣ Невозможно управлять правилами

Сегодня, чтобы изменить правило:

- нужно менять код
- деплоить сервис
- согласовывать релиз
- рисковать продом

Бизнес ненавидит это.

Decision Hub позволяет:

- менять правила без изменения бизнес-сервисов
- версионировать решения
- делать A/B решений
- анализировать последствия

💰 Это скорость бизнеса.

🎯 В чём ценность слоя — одной фразой

> **Он превращает “хаос проверок” в управляемое бизнес-решение.**

🧩 Очень простой пример (без IT слов)

Было:

> «Операция не прошла, потому что где-то что-то не так.»

Стало:

> «Операция не прошла, потому что:
>
> - правило AML-102 — сработало
> - риск 0.91
> - лимит превышен на 12%»

Это **другой уровень управления**.

🧠 Почему без этого слой кажется “лишним”

Потому что сейчас:

- решения живут в коде
- код пишут разработчики
- бизнес не владеет логикой
- аналитика невозможна

Но пока не было боли — никто не задумывался.

📊 Что получает бизнес конкретно

Ценность	Что меняется

Прозрачность	понятно, почему отказ
Контроль	видно, какие правила работают
Управляемость	можно менять правила
Скорость	не ждать релиз
Аудит	можно показать регулятору
Деньги	меньше false positive

Это **прямая бизнес-ценность**, не IT-игрушка.

⚠️ Важный момент

Decision Hub **НЕ нужен**, если:

- маленький продукт
- нет регулятора
- нет AML
- нет антифрода
- нет лимитов
- нет аудита

Но в банке:

➡️ он появляется **естественно**, просто сейчас он размазан.

🧠 Самая сильная мысль

> Сейчас Decision Hub уже существует.
> Просто в худшей возможной форме — в коде.

Ты не предлагаешь новый слой.

Ты предлагаешь: **вытащить решения из кода в управляемую модель.**

> «Проблема не в том, что у нас много сервисов.
> Проблема в том, что решение не имеет владельца.»

🔥 Главный тезис

> **Бизнес-сервисы не содержат правил.**
> **Они только исполняют результат.**

А правила выносятся в **отдельный Decision Layer**, который:

- получает входные данные
- применяет правила
- возвращает решение
- логирует «почему»

❓ Что значит «менять правила без изменения бизнес-сервисов»

Это НЕ значит:

- «правки без разработчиков»
- «бизнес сам крутит кнопки»

Это значит:

> ❗ бизнес-сервис НЕ знает формулы, условий и if-ов

Он знает только:

```
decision = decision_client.evaluate(context)

if decision.allowed:
    execute()
else:
    reject(decision.reason)
```

ВСЁ.

🧠 Где раньше были правила

AS-IS (сейчас)

```
if (amount > limit) {
  return LIMIT_ERROR;
```

```
}

if (riskScore > 0.8) {
  return FRAUD_BLOCK;
}

if (country in blockedCountries) {
  return AML_REJECT;
}
```

❌ правила в коде
❌ каждый if = релиз
❌ нет версий
❌ нет истории
❌ нет объяснения

✅ TO-BE (Decision Layer)

```
DecisionResult result =
  decisionHub.evaluate("PAYMENT", context);

if (!result.allowed()) {
  return result.error();
}
```

Сервис **не знает**, какие правила внутри.

🧩 Как это работает архитектурно

```
PaymentService
  |
  | DecisionRequest (JSON)
  ↓
Decision Hub
  |
  ├── Rule Engine
  ├── Rule Repository
  ├── Versioning
  ├── Explanation Engine
  ↓
DecisionResult
```

🧠 Что такое «правила» технически

Это не магия.

Это обычные структуры данных.

Пример правила:

```
{
 "rule_id": "AML_102",
 "condition": "country in blocked_countries",
 "action": "REJECT",
 "severity": "HIGH",
 "version": "3.2"
```

```
}
```

Или:

```
{
  "rule_id": "LIMIT_DAILY",
  "condition": "daily_sum + amount > 10_000_000",
  "action": "REJECT"
}
```

Они:

- лежат в БД
- версионируются
- активируются / деактивируются
- не требуют пересборки сервиса

⚙️ Как Rule Engine применяет правила

Псевдокод:

```
rules = repository.load_active_rules("PAYMENT")

for rule in rules:
    if rule.match(context):
        log(rule)
        if rule.action == REJECT:
            return DecisionResult.reject(rule)
```

Это всё.

Никакой магии.

🧠 Что меняется принципиально

Было	Стало
if в коде	данные в БД
релиз	конфигурация
лог «ошибка»	лог «правило №102»
невозможно объяснить	можно объяснить
нет истории	есть аудит
хаос	модель

🔍 Как выглядит контекст (что передаётся)

```
{
 "client_id": "123",
 "amount": 500000,
```

"currency": "UZS",
"country": "IR",
"device_trust": "LOW",
"daily_sum": 9500000
}

Decision Hub **не дергает БД сервисов**.

Он работает с **контекстом**, который ему передали.

Это важно.

⚠️ Важнейший архитектурный принцип

> Decision Hub не владеет данными.
> Он владеет логикой.

Данные приходят извне.
Решения принимаются внутри.

🧠 Почему это безопасно

- сервисы не ломаются
- API не меняется
- формат ответа стабилен

- меняются только правила

Это как менять тарифы, не переписывая биллинг.

🧩 Пример реального изменения без релиза

Было правило:

LIMIT_DAILY = 10 млн

Бизнес решил:

LIMIT_DAILY = 5 млн

Что происходит?

- меняется запись в БД
- правило версии 3.3 активируется
- сервисы даже не знают об этом

Никакого деплоя.

🔥 Где тут реальная бизнес-ценность

1️⃣ быстрее реакция на фрод
2️⃣ меньше простоев
3️⃣ меньше релизов
4️⃣ меньше ошибок
5️⃣ объяснимые отказы
6️⃣ регулятор доволен

Это **не IT-выгода**.
Это **операционная выгода банка**.

⚠️ Очень важно (честно)

Decision Hub **НЕ нужен для всех кейсов**.

Он нужен только там, где:

- есть правила
- есть риск
- есть регулятор
- есть деньги
- есть ответственность

P2P, платежи, кредиты — да.
Просмотр баланса — нет.

🎯 Теперь главное

Ты сейчас понял самую важную мысль:

> мы не добавляем слой
> мы вытаскиваем правила из кода

И вот здесь рождается зрелая архитектура.

Таким образом, я хочу чтобы мы могли создавать проекты для процессов **P2P, loans, deposits, transfers, BNPL, скоринг, кредитного бюро (кредитная история) и так далее**:

- разобрать **конкретный кейс P2P AS-IS → TO-BE**
- показать **реальный JSON контекста**
- показать **как выглядят версии правил**
- показать **как это логируется (trace + ruleId)**

🧠 Ключевая мысль

❗ Децентрализация ≠ блокчейн

Блокчейн — это **технический механизм**.

А децентрализация — это **организационный и архитектурный принцип**.

Можно:

- быть децентрализованным без блокчейна

- и быть централизованным с блокчейном

(в реальности — так чаще и бывает)

🧩 Что такое децентрализация в инженерном смысле

> **Отсутствие единого владельца логики принятия решений.**

Не «серверов»,
не «нод»,
а именно — **логики**.

Пример централизованной системы

```
PaymentService
  ├── antifraud rules
  ├── limit rules
  ├── aml rules
  └── risk rules
```

Один сервис = один владелец = одна точка истины.

Это **централизация**, даже если сервисов много.

Пример децентрализованной логики

```
Decision Hub
  ├── Limit Engine (владелец: Финконтроль)
  ├── Fraud Engine (владелец: Fraud)
  ├── AML Engine (владелец: Compliance)
  └── Risk Engine (владелец: Risk)
```

Каждый модуль:

- имеет своего владельца
- обновляется независимо
- имеет свои правила
- имеет свою версионность

👉 это уже **децентрализация логики**,
даже если всё работает в одном датацентре.

🔥 Это и есть настоящая децентрализация в финтехе

Не ноды.

А **независимые центры принятия решений**.

🧠 Почему блокчейн тут вообще всплывает

Потому что блокчейн решает три задачи:

1. неизменяемость (immutability)
2. доверие между сторонами
3. независимая верификация

Но в банке:

- доверие уже есть (юридическое)
- участники известны
- регулятор сверху

Поэтому блокчейн **не обязателен**.

🧩 Тогда зачем он вообще может понадобиться?

Только если появляется **внешнее доверие**:

- несколько банков
- финтех-партнёры
- маркетплейсы
- BNPL провайдеры
- cross-bank решения

Тогда возникает вопрос:

> ❓ кому верить при принятии решения?

И вот здесь блокчейн начинает иметь смысл.

🔗 Как Decision Hub может эволюционировать

Этап 1 — без блокчейна (реально и правильно)

- обычные сервисы
- REST / gRPC
- БД
- audit log
- versioning

Это **100% рабочая финтех-модель**.

И именно её ты можешь показать коллегам.

Этап 2 — федеративная модель (очень важный шаг)

Bank A → Decision Module
Bank B → Decision Module
Fraud Provider → Decision Module

Каждый предоставляет **свой расчёт**.

Решение агрегируется.

Это уже **децентрализация между организациями**,
но всё ещё без блокчейна.

Этап 3 — blockchain (опционально)

Блокчейн нужен ТОЛЬКО когда:

- участники не доверяют друг другу
- нельзя изменить историю решений
- нужен публичный аудит
- нужна независимая верификация

Пример:

> «этот платёж был разрешён консенсусом модулей»

Но это **не стартовая точка**.

🔍 Очень важный момент (и тут твой код как раз в тему)

```
DecisionResult result =
    decisionHub.evaluate("PAYMENT", context);
```

Это **точка абстракции**.

Она не говорит:

- где живёт Decision Hub
- как он реализован
- централизован он или распределён

Сегодня:

- это REST вызов

Завтра:

- federated call
- message bus
- consensus
- blockchain proof

Код бизнес-сервиса **не меняется вообще**.

Вот в чём сила идеи.

🧠 Это называется architectural optionality

Ты создаёшь архитектуру, где:

- блокчейн возможен
- но не обязателен
- и не мешает сейчас

Это признак зрелого архитектора.

💡 Очень простой пример для коллег

Можно объяснять так:

> «Мы не строим блокчейн.
> Мы строим слой решений.
> А блокчейн — возможный способ их фиксировать, если понадобится.»

Это сразу снимает страх.

🎯 И вот главный инсайт

Ты не предлагаешь Web3.

Ты предлагаешь:

> **отделить решение от исполнения.**

Это работает:

- в банке
- без крипты
- без токенов
- без хайпа

А децентрализация — это **следствие**, а не цель.

🧠 Финальный ответ

✔ возможна ли идея без блокчейна?

Да. И именно так она должна начинаться.

✔ связана ли она концептуально с децентрализацией?

Да — на уровне логики и владения решениями.

✔ обязателен ли блокчейн?

Нет. Он — опциональный механизм доверия, а не фундамент.

🧠 1) Что вообще считается “стандартом” в банках сейчас

В традиционных банковских системах почти всегда есть:

✅ сервисы, которые принимают решение
— например, антифрод, лимиты, KYC/AML, риск-оценка
✅ много проверок внутри бизнес-сервисов
— код платежа сам проверяет лимиты, риск и т. д.
✅ аудит и трассировка
— но часто распределённые и разрозненные

То есть архитектурно:

```
PaymentService
  ├── if antifraud
  ├── if limit
  ├── if risk
  ├── if AML
  └── return decision
```

Это **не считается архитектурно зрелым**, но это есть почти везде.

🧠 2) Актуальность выделенного слоя решений

То о чём мы говорим — то есть выделенный **Decision Layer / Decision Hub / Rule Engine** — в банковской архитектуре встречается всё чаще, и не только в топ-банках.

Это **не стандартизировано как обязательный модуль**, но это:

- 🔹 **узнаваемая архитектурная практика**
- 🔹 часть зрелых реализаций
- 🔹 часто встречается в крупных системах

Часто это называют по-разному:

- Decision Service / Decision Engine
- Business Rules Engine (BRE)
- Compliance Decision Layer
- Risk Orchestration Layer
- Policy Decision Point (PDP) в терминах XACML

🧠 3) По каким причинам банки это делают

📌 Боль 1 — неизменяемая логика и частые изменения правил

Бизнес хочет чаще менять правила без роллаута сервиса.

Решение:

бизнес меняет правило в Decision Hub
→ сервисы просто читают готовое решение
→ нет релиза

Это уменьшается время отклика бизнеса.

📌 Боль 2 — объяснимость и аудит

Регуляторы требуют:

❓ почему операция была разрешена/запрещена?
❗ каковы аргументы этого решения?

Если правила находятся внутри кода:

- ответ «потому что код так написал разработчик»
- нет истории
- нет версии правил

Если правило вынесено отдельно:

- можно сказать точно:
  “отказ по правилу AML-117 версии 4.2”
- есть журнал применения
- есть версия
- есть владелец

Это **реальная регуляторная ценность**.

📌 Боль 3 — переиспользование логики

Один и тот же набор правил может использоваться:

- в мобильном приложении
- в API-слое
- в batch-процессах
- в интеграциях с партнёрами

Если логика в коде сервисов — она дублируется.

Если она вынесена — её можно переиспользовать.

Это **engineering efficiency**.

🧠 4) Это *уже часть практики*, но не массово

На практике банки идут примерно так:

Банк	Decision Hub
крупный международный	есть
крупный национальный	части есть
средний банк	частично, ad hoc
малый банк	нет

То есть:

🎯 **чем сложнее сервисы, чем выше требования по комплаенсу, рискам и ответам регулятора — тем более зрелая архитектура**
➡ и тем чаще встречается выделенный слой правил/решений.

🧠 5) Где это уже есть (примерные реальные механизмы)

✅ Policy/Rules Engine
– Drools / Opensource BRMS
– IBM ODM / FICO Blaze
– Decision Services на BPM/Automation

✅ Orchestration Service
– Camunda / Zeebe / Temporal

✅ Compliance Decisions
– отдельные микросервисы, вынесенные из ядра

Но при этом:

❗ это не обязательно «Decision Hub V1.0» как единый продукт
❗ это может быть смесь модулей

🧠 6) Что это *не является обязательным стандартом*

В банковской архитектуре **нет жесткого стандарта**, который требует:

> «У всех банков должен быть Decision Hub с отдельным слоем правил и версионностью».

Но есть **паттерн архитектурной зрелости**:

👉 сначала логика внутри сервисов
👉 потом — вынесение Business Rules
👉 потом — Orchestration
👉 потом — Shared Decision Layer

Это как ступени зрелости архитектора/банка.

🧠 7) Почему это *важно именно сейчас*

Сейчас в финтех и банковской архитектуре есть несколько трендов:

Тренд 1 — microservices

→ правила в разных сервисах создают разрозненность

Тренд 2 — regulatory demand

→ объяснимость решений

Тренд 3 — DevOps / CI/CD

→ бизнес хочет менять правила без релизов

Тренд 4 — observability

→ trace + audit + explainable decisions

Все они логично ведут к выделенному решению уровня:

решение = данные (контекст) + правила (external) → результат + объяснение

🧠 8) Что это *не обязательно* делает блокчейн

Блокчейн — это отдельная история про независимые доверенные участки без общей власти.

В банковских системах:

- участники известны
- доверие есть юридически
- доверие к данным — через audit logs и SIEM

Поэтому начало всегда:

> **без блокчейна, только через обычные сервисы и правила**

⚙️ 9) Итак: как это выглядит в архитектуре NOW

```
API Gateway
    ↓
Business Service A
    ↓
Decision Service
   / | \
Limit Fraud AML Risk
```

Decision Service – это централизованная точка принятия решения.

Но нет ещё:

```
Decision Service (Bank A)   Decision Service (Bank B) ...
    \                           /
        federated or cross-bank layer
```

А вот это уже элемент **децентрализации между организациями**.

🧠 10) Как это выглядит с точки зрения кодовой архитектуры

Без Decision Layer (традиционно)

```
if (amount > dailyLimit) return reject;
if (risk > threshold) return reject;
if (blockedCountry.contains(country)) return reject;
```

С Decision Layer

```
DecisionResult decision =
  decisionClient.evaluate(context);

if (!decision.allowed()) {
   return reject(decision.reason());
}
```

```
Decision Layer:

rules = ruleRepo.loadRules("payment")
for rule in rules:
   if rule.matches(context):
      return decisionFrom(rule)
```

return allowedDecision

И:

rules stored in DB
rule versions
audit trail
owner info
explanation
metrics

Это уже **enterprise-уровень**.

📌 Итог — в твоих словах

❗ отделение *логики решений* от *исполнения* —
актуальная практика зрелых банковских архитектур.

❗ это не обязательно уже есть в ТЗ/коде —
часто логика остаётся разбросанной.

❗ так оно должно быть *в больших системах и платёжных потоках*.

❗ это **не стандарт по регламенту**, но **архитектурный паттерн зрелого банка**.

🧠 Главное: вычисления ≠ математика

Когда мы говорим **«вычисления»** в финтехе, мы почти никогда не имеем в виду:

- интегралы
- нейросети
- GPU
- сложную математику

Вычисления в финтехе — это:

> **преобразование контекста → в решение**

И это **очень тяжёлый вычислительный процесс**, просто не в математическом смысле.

🧩 Что реально «вычисляется»

1️⃣ Decision Score (самое важное)

Решение — это не просто `true / false`.

Это почти всегда:

```
{
 "decision": "REJECT",
 "confidence": 0.92,
 "reasons": [
  "AML_102",
```

"FRAUD_017"
]
}

То есть мы вычисляем:

- итоговое решение
- степень риска
- набор факторов
- приоритет факторов

Это уже **агрегация вычислений**.

🔍 Пример из реальности банка

Вход:

client
amount
device
geo
history
time
frequency
merchant

Каждый модуль возвращает:

Модуль	Результат
Limit	OK
AML	WARNING
Fraud	RISK=0.87
Risk	MEDIUM

Decision Hub должен вычислить:

- какой вес важнее
- что перекрывает что
- что является стоп-фактором
- какое правило имеет приоритет

👉 это **вычисление композиции решений**.

🧠 Это и есть compute

```
final_decision =
   f(limit, fraud, aml, risk, policy)
```

Это не if-else.

Это **функция агрегации решений**.

🧩 Типы вычислений в Decision Hub

Теперь конкретно.

🟢 1. Rule evaluation (база)

Да, сначала — правила.

Но:

- условия
- комбинации
- зависимости
- приоритеты
- override
- stop-factors

Это не «один if».

Пример:

IF fraud_score > 0.8 AND amount > 100k
THEN REJECT

Это уже вычисление.

🟢 2. Rule chaining (цепочки)

Решение может запускать другое правило.

Например:

```
IF AML = HIGH
 → increase fraud sensitivity
```

То есть одно правило меняет вход для другого.

Это уже **динамическое вычисление контекста**.

🟢 3. Weighted scoring

Очень часто решение — это сумма весов:

```
risk_score =
   fraud_score * 0.5 +
   device_score * 0.3 +
   geo_score * 0.2
```

Это **реальные вычисления**, просто CPU, не GPU.

🟢 4. Threshold evaluation

if risk_score > 0.85 → reject

if 0.6–0.85 → challenge
else → approve

Это вычисление диапазонов.

🟢 5. Conflict resolution

Если:

- AML says BLOCK
- Fraud says ALLOW

Кто важнее?

Decision Hub вычисляет **приоритет источников**.

Это бизнес-математика.

🟢 6. Temporal computations

Например:

- 3 операции за 10 минут
- сумма за сутки
- частота за период
- аномалия поведения

Это уже stateful-вычисления.

🟢 7. Explainability computation

Decision Hub вычисляет:

- какие правила реально повлияли
- какие были проверены
- какие отброшены
- почему принято именно это решение

Это отдельная логика.

🧠 Почему это реально compute, а не конфигурация

Потому что:

- входные данные каждый раз разные
- результат не предсказуем заранее
- порядок правил меняет результат
- веса меняют результат
- комбинации экспоненциальные

Это **динамическое вычисление решения**, а не статическая проверка.

🔥 Самый важный момент

> Мы вычисляем не правило.
> Мы вычисляем **решение как результат множества независимых сигналов**.

И вот здесь аналогия с GPU становится честной:

GPU compute	Fintech decision compute
тысячи операций	десятки сигналов
параллельно	параллельно
агрегация	агрегация
финальный результат	финальное решение

🧩 Почему это нельзя оставлять в бизнес-сервисе

Потому что:

- код превращается в ад
- if-else растут экспоненциально
- невозможно объяснить результат
- невозможно тестировать комбинации
- невозможно управлять приоритетами

Decision Hub — это **движок вычислений решений**.

🎯 И теперь самая точная формулировка

> **Decision Hub — это вычислительный слой бизнес-решений.**

Он вычисляет:

- не деньги
- не балансы
- не транзакции

А:

> ✔ риск
> ✔ допустимость
> ✔ приоритет
> ✔ объяснение
> ✔ ответственность

🧠 Поэтому это и compute

Не AI-compute.
Не GPU-compute.

А **decision compute**.

И в банках это один из самых дорогих видов вычислений —
потому что от него зависит деньги, регулятор и репутация.

Тезис:

> **«Вычисления решений (decision compute) — это один из самых дорогих видов вычислений в банках, потому что от него зависит деньги, регулятор и репутация.»**

Такое утверждение **требует обоснования**, потому что в индустрии нет формального стандарта, который говорит именно так. Но есть **несколько независимых источников и практик**, на которых базируется эта оценка.

Мы разберём:

1. Откуда берётся практика отделения логики принятия решений
2. Откуда берутся оценки важности/стоимости этой логики
3. Какие реальные подходы и стандарты это подтверждают
4. Почему банки так делают — не просто “потому что модно”

🧩 1) Откуда берётся паттерн *decision services / rule engines*

Существует **много практик и стандартов**, которые выделяют логику принятия решений в отдельный слой:

✅ **Business Rules Management Systems (BRMS)**
— это признанная категория ПО, включающая Drools, IBM ODM, FICO Blaze, Red Hat Decision Manager, Oracle Policy Automation и др.

BRMS — это не теория, это **продукты для управления бизнес-правилами в отдельном слое**, которые применяются в банковских системах, страховании, логистике и ритейле.

💡 Сами BRMS появились как ответ на проблемы:

- рост дублирования логики
- сложность сопровождения
- постоянные изменения правил
- отсутствие traceability

📌 Это подтверждается практическими описаниями архитектур, например в:

- **Gartner — Business Rules Management report**
- **Forrester — Decision Management Platforms**
- **IBM, Red Hat, FICO, Oracle documentation** по BRMS/BRE

Во всех них выделение правил отдельно — не “мода”, а **архитектурная рекомендация**.

🧩 2) Почему это считается дорогостоящей частью системы?

Дорогостоящей по четырём причинам:

📌 Причина 1 — высокая частота изменений

Правила в финтехе меняются часто:

- лимиты
- комплаенс
- AML/CTF правила
- правила скоринга

- риск-факторы

Если логика в коде — каждый раз новый релиз.

Если логика вне кода — изменения можно делать быстрее, безопаснее, с audit trail.

💡 Это именно то, что BRMS и Decision Management предлагают.

📌 Причина 2 — регуляторная ответственность

Регуляторы (ЦБ, FATF/FSB, AMLD, PCI DSS и др.) требуют:

- объяснимость решений
- audit trail
- документируемые правила
- версия правил

Это прямо указано в регуляторных требованиях.

Примеры:

🔹 **AML/CTF frameworks** (FINRA, FCA, FATF)
— требуют объяснимости, traceability

🔹 **PCI DSS**
— требует контроля изменений, аудита

Это не домыслы — это часть регуляторных чеклистов.

📌 Причина 3 — комплаенс и судебная защита

В случае спорных транзакций у банка должны быть:

- что именно проверялось
- на основании каких правил
- версия правила
- лог применения

Если этого нет — банк несёт:

- финансовые санкции
- репутационный риск
- юридические потери

Это дорого — не в CPU, а в **бизнес рисках**.

📌 Причина 4 — масштабирование и переиспользование

В распределённых системах:

- правила дублируются
- правила работают по разным каналам
- разные команды пишут одно и то же

Выделенный слой экономит время разработчиков и снижает баги.

📌 Почему это НЕ “дорого” вычислительно

Важно уточнить:

> когда говорят **“дорого”**, это НЕ значит, что вычисления тяжелы для процессора.

Это значит:

📍 дорого в:

- **engineering effort**
- **compliance effort**
- **operational risk**
- **audit / governance**
- **time to change**

То есть это **стоимость управления**, а не просто CPU cycles.

🧩 3) Что говорят профессиональные источники

Существует ряд независимых источников, подтверждающих архитектурную ценность decision services:

📌 Gartner (Decision Management + BRMS)

Gartner выделяет направление **Decision Management**
— области, где управляемые правила и логика принятия решений выносятся из кода в отдельные компоненты.

Именно для:

- финтех
- страхования
- кредитования

- маркетинга

Это признанный архитекторский паттерн.

📌 Forrester (Decision Platforms)

Forrester анализирует **Decision Platforms** как совокупность:

- business rules
- analytics
- AI/ML scoring
- rule authoring
  Это отражает индустриальное движение к **выделению логики**.

📌 DMN (Decision Model and Notation)

Открытый стандарт от OMG (Object Management Group):

📍 **DMN — это стандарт описания решений**, который используется именно для того, чтобы отделить:

- данные
- правила
- модели решений

Это не академическая теория — это стандарт, поддерживаемый многими продуктами.

🧩 4) Где это уже внедрено

Такой подход не просто теоретический — он находится в промышленном применении:

🔹 крупные банки (JP Morgan, Citi, HSBC)
— используют BRMS и centralized decision services

🔹 платёжные провайдеры (Stripe, Adyen)
— используют решения, отделённые от core services

🔹 финтехы (Revolut, Wise)
— построили правила отдельно, чтобы быстро менять

🧠 5) Почему это выгодно бизнесу (не инженерам)

Важно понять:

> ценность — не в “вычислении CPU”, а в **ускорении бизнеса и снижении рисков**.

Четыре примера:

📌 Пример 1 — изменение правила

Бизнес хочет изменить лимит с 10 млн → 5 млн.

Если правило в коде:

- код пишется
- тестируется
- деплоится
- может сломать что-то

Если правило вынесено:

- бизнес изменяет правила
- логируется
- история версий
- отслеживаемость

Второй вариант дешевле по effort.

📌 Пример 2 — объяснимость решения для регулятора

Регулятор спрашивает:

> почему транзакция была отклонена?

Если правило в коде —
ответ “потому что код такой”.

Если правило в Decision Engine —
ответ со всеми доказательствами.

Более сильная позиция для банка.

📌 Пример 3 — централизация правил

Если одно и то же правило используется в:

- мобайл
- API
- batch
- партнёрские интеграции

decision layer позволяет избежать дублирования.

Это экономит Engineering time.

📌 Пример 4 — трассировка и audit

Если действия разбросаны по 10 сервисам:

- лимиты
- AML
- antifraud
  → выдача

То интегрировать все логи в один ответ — сложная реляция.

Если логика вынесена — trace готов.

🧠 6) Почему я сказал “одно из самых дорогих”

Потому что:

📍 все перечисленные пункты —
это фактическая бизнес-стоимость, и она часто выше стоимости самих транзакций.

Вместо:

💸 выкатить новый сервис
банк платит:

- ошибки правил
- регуляторные штрафы
- задержки релизов
- человеческие расследования
- поддержку ручного аудита

📌 Формулировка, которую можно использовать

> **“Это не вычислительная нагрузка, это вычислительная ответственность.**
> **Decision compute — это не тяжёлые числа, это тяжёлая логика принятия решений,**
> **и банки за неё платят не CPU, а effort, time-to-change, compliance cost и audit cost.”**

🧾 Источники / категории подтверждения

Источник	Что подтверждает
Gartner (Decision Management)	выделение логики решений

Forrester (Decision Platforms)	value of decision logic
DMN Standard	архитектурный паттерн
BRMS products documentation	practical implementations
regulatory frameworks (AML/CTF, PCI DSS)	need for explainability
industry case studies	real banking practices

🧭 Реальная архитектура (как она есть)

Упрощённо сейчас у нас примерно так:

Mobile App
↓
MBGW (API Gateway)
↓
Business Service (Payments / P2P / etc)
↓
AnorHub (ESB / integration)
↓
Core / External systems

Где-то сбоку:

- антифрод
- лимиты
- проверки
- AML
- ручные кейсы
- Camunda (частично)

И главная проблема:

> ❌ решение «можно / нельзя» не принадлежит ни одному слою

🔥 Где сейчас живёт решение (AS-IS)

Фактически сейчас решение живёт:

- немного в Payment Service
- немного в Antifraud
- немного в Core
- немного в Camunda
- немного в голове аналитика

То есть:

РЕШЕНИЕ = сумма побочных эффектов

Это худшая форма архитектуры.

🎯 Что мы хотим получить

Мы хотим, чтобы **решение стало артефактом**.

Не side-effect.

А объектом:

```
DecisionResult {
  allowed: false,
  reasons: ["AML_102", "FRAUD_017"],
  risk_score: 0.91,
  trace_id: "abc-123"
}
```

И вот здесь появляется Decision Hub.

🧩 Где Decision Hub в вашей архитектуре

Очень важно:

> ❗ Decision Hub — НЕ новый Gateway
> ❗ НЕ замена MBGW
> ❗ НЕ ESB

Он встаёт **между бизнес-сервисом и оркестрацией**.

TO-BE:

Mobile
↓
MBGW
↓
Payment Service
↓
Decision Hub
↓
Camunda (если нужен процесс)
↓
AnorHub / Core

🧠 Почему именно тут

Потому что:

- MBGW — транспорт + безопасность
- Payment Service — владелец операции
- Decision Hub — владелец решения
- Camunda — владелец процесса
- AnorHub — владелец интеграций

Каждый слой отвечает за **своё**.

Это и есть зрелая архитектура.

🔍 Как это работает пошагово (реальный P2P)

Шаг 1. Клиент инициирует перевод

POST /p2p/transfer

MBGW делает:

- JWT
- device
- rate limit
- headers
- correlation-id

❗ MBGW НЕ принимает решений.

Шаг 2. Payment Service собирает контекст


```
{
 "client_id": "123",
 "amount": 500000,
 "currency": "UZS",
 "receiver": "456",
 "device_trust": "LOW",
 "daily_sum": 9500000,
 "channel": "MOBILE"
}
```

Payment Service делает свою работу:

- сбор данных
- нормализация
- enrichment

❗ Он всё ещё не решает.

Шаг 3. Вызов Decision Hub

POST /decision/evaluate

```
{
  "decision_type": "P2P_TRANSFER",
  "context": {...}
}
```

Это ключевая точка.

🧠 Что делает Decision Hub внутри

Он запускает вычисления:

Параллельно:

- Limit Engine
- Fraud Engine
- AML Engine
- Risk Engine

Каждый возвращает:


```
{
  "module": "FRAUD",
  "result": "RISK",
  "score": 0.87,
  "rule": "FRAUD_017"
}
```

Потом Decision Hub вычисляет:

- приоритеты
- веса
- стоп-факторы
- override правила

И формирует:


```
{
  "allowed": false,
  "decision": "REJECT",
  "reasons": [
    "FRAUD_017",
    "AML_102"
  ],
  "risk_score": 0.91
```

}

⚠️ ВАЖНО:
это решение **уже объяснимо**.

🧠 Где тут Camunda

Camunda **НЕ должна принимать решение**.

Она должна:

- оркестрировать действия ПОСЛЕ решения
- управлять флоу
- делать компенсации
- управлять асинхронностью

Например:

если decision = APPROVE
→ start payment saga

если decision = CHALLENGE
→ start verification process

если decision = REJECT
→ finish flow

👉 Camunda = процесс
👉 Decision Hub = логика

Это принципиально.

🔥 Это очень важное разделение

Компонент	Ответственность
MBGW	безопасность / вход
Payment Service	операция
Decision Hub	решение
Camunda	процесс
AnorHub	интеграция

Если смешивать — появляется ад.

🧠 Где тут AnorHub (ESB)

AnorHub должен жить **ниже решения**.

Потому что:

- интеграции не должны влиять на решение
- внешняя система не должна решать “можно или нельзя”

Decision сначала.
Интеграции — потом.

Это критично для устойчивости.

🔍 Почему это резко упрощает архитектуру

Сейчас:

- каждый сервис думает
- каждый сервис решает
- каждый сервис ошибается

Становится:

- сервис спрашивает
- decision layer отвечает
- ответственность централизована

💰 Где реальная бизнес-ценность

Теперь бизнес может:

- видеть все причины отказов
- менять правила централизованно
- проводить анализ false positive

- защищаться перед регулятором
- объяснять клиенту причины

Это не теория.
Это операционное преимущество.

🧠 Почему это децентрализация (в реальности)

Потому что:

- лимиты владеет финблок
- антифрод владеет риск-блок
- AML владеет комплаенс
- Payment не владеет ничем из этого

Каждый модуль автономен.

Это **децентрализация логики**, а не серверов.

> **Децентрализация не в месте исполнения,**
> **а в месте владения логикой.**

Исполнение может быть централизованным.
Владение — нет.

🧠 Где я сейчас вижу «централизацию»

Ты видишь вот это:

Payment Service
  ↓
Decision Hub ← один сервис

И логично думаешь:

> «Один сервис = центр = централизация»

И если смотреть **только на инфраструктуру** — ты прав.

Но децентрализация **не про количество сервисов**.

🧩 Что такое настоящая децентрализация (без блокчейна)

Децентрализация =

> отсутствие единого владельца правил и решений

Не единой точки вызова —
а единой точки **владения смыслом**.

⚠️ Важное различие

Что	Может быть централизовано	Не должно быть централизовано
Исполнение	✅	
API	✅	
Runtime	✅	
Владение логикой		❌
Правила		❌
Решения		❌

Decision Hub централизует **исполнение**,
но **децентрализует смысл**.

🧠 Как это выглядит на практике

Внутри Decision Hub нет “главного правила”.

Есть независимые домены:

```
Decision Hub
├── Limit Domain     (владелец: финконтроль)
├── Fraud Domain     (владелец: риск)
```

```
├── AML Domain      (владелец: комплаенс)
├── Risk Domain     (владелец: риск-менеджмент)
```

Каждый:

- имеет своего владельца
- имеет свои правила
- имеет свою модель
- имеет свою версионность
- деплоится отдельно (или логически отдельно)

🔥 Вот здесь и есть децентрализация

Не один “мозг”,
а несколько **независимых центров принятия решений**.

🧠 Аналогия (очень точная)

Конституционный суд

- есть один зал заседаний
- есть один протокол

Но:

- судья по уголовным делам ≠ судья по гражданским
- каждый принимает решение независимо
- итог — композиция решений

Исполнение централизовано.
Власть — распределена.

🧩 Почему это не противоречие

Потому что:

- централизовано **выполнение**
- децентрализована **ответственность**

Это абсолютно нормально в сложных системах.

🔍 Что было раньше (AS-IS)

Раньше:

```
Payment Service
├── limit logic
├── fraud logic
├── aml logic
└── risk logic
```

Один сервис владеет всем.

Это **настоящая централизация**.

✅ Что стало (TO-BE)

```
Payment Service
  ↓
Decision Hub
    ├── Limit rules (owner A)
    ├── Fraud rules (owner B)
    ├── AML rules (owner C)
    └── Risk rules (owner D)
```

Payment Service больше не владеет логикой.

Это и есть децентрализация.

🧠 Очень важная формулировка (её можно цитировать)

> **Мы не децентрализуем инфраструктуру.**
> **Мы децентрализуем право принимать решение.**

И это в банке куда важнее блокчейна.

🎯 Почему это близко к твоей философии децентрализации

Потому что ты говоришь не про:

- ноды
- токены
- консенсус

А про:

- автономию команд
- ответственность
- границы доменов
- отсутствие “сервиса-бога”

Это **domain decentralization**, а не crypto decentralization.

🧠 Где здесь потенциально появляется блокчейн (позже)

Только если:

- владельцы логики — разные юрлица
- нет доверия
- нужен внешний аудит

Тогда:

- execution может быть federated
- логика подписана

- решения зафиксированы

Но это **вторая фаза**, не первая.

🧩 Финальный вывод

Ты сейчас можешь очень чётко сказать команде:

> «Мы не хотим один умный сервис.
> Мы хотим убрать монополию одного сервиса на решения.»

Decision Hub — не центр власти.
Он — **место встречи независимых решений**.

✅ Что реально можно сделать так, чтобы банк мог это использовать

1) Reference implementation (эталонный проект)

Это самый реалистичный формат.

Что получит команда:

- понятную структуру сервисов
- контракты API (OpenAPI)
- примеры интеграций (sync + async)
- логирование/trace/correlation-id

- пример Decision Hub + объяснимость
- примеры ошибок/ретраев/идемпотентности
- Docker Compose для запуска

Это можно использовать:

- как шаблон для новых сервисов
- как обучающий стенд для CA и разработчиков
- как основу для ADR (“почему так лучше”)

2) “Архитектурный симулятор банка”

Ты можешь смоделировать **похожую топологию**, но без реальных данных и секретов:

- Mobile (mock)
- API Gateway (упрощённый)
- Payment service
- Decision Hub
- 2–3 “модуля” (Limits, Fraud, AML)
- “Core” (упрощённый ledger)
- Event bus (Kafka/Rabbit) по желанию
- БД на Postgres

Важно: это будет **аналог**, а не копия банка.

✅ Можно ли “в коде подсветить почему это лучше”

Да, и это делается 3 способами (очень эффективно):

A) Трассировка и explainability “из коробки”

Ты показываешь коллегам не словами, а артефактом:

- `trace_id`
- `decision_id`
- `rule_id`
- причины отказа в одном ответе
- журнал применения правил

Прямо в ответе API:

```
{
  "status": "REJECTED",
  "trace_id": "…",
  "decision": {
    "allowed": false,
    "reasons": ["AML_102", "FRAUD_017"],
    "risk_score": 0.91
  }
}
```

B) Сравнение AS-IS vs TO-BE через два эндпойнта

- `/p2p/transfer-legacy` — логика размазана
- `/p2p/transfer` — через Decision Hub

И на демонстрации ты показываешь:

- где проще менять правила
- где лучше аудит
- где меньше зависимостей
- где выше тестопригодность

C) Тесты как доказательство

Ты добавляешь “правила как данные” и тесты типа:

- “если AML_102 активен — блок”
- “если лимит изменили — сервис не меняется”
- “если риск 0.9 — challenge”

Тесты — лучший аргумент для разработчиков.

✅ Можно ли выстроить похожие сервисы и БД как в банке

Да, **но правильно — через “каноническую модель”**, а не через копирование:

- в проекте будут **таблицы-аналоги**
- **минимальный набор сущностей**
- **те же доменные связи**
- но без чувствительных полей и секретов

Пример:

- `accounts`, `balances`
- `transfers`
- `limits`
- `decisions`, `decision_rules`, `decision_audit`
- `events_outbox` (если показываешь event-driven)

Это будет очень похоже на банковскую реальность и отлично читается коллегами.

⚠️ Что важно, чтобы проект реально “зашёл” в банк

1) Security/Compliance-ограничения

Чтобы банк мог брать код:

- никаких секретов в репо
- конфиги через env
- примеры маскирования логов
- простая модель ролей/доступов

2) Технологии, которые “примут”

Если у банка Java/.NET — ты можешь сделать демо на Python, но:

- структура и паттерны должны быть переносимы
- контракты и архитектура важнее языка

Если хочешь максимальную “прикладную ценность”, делай:

- либо на том стеке, что у банка
- либо на Python/FastAPI, но с идеальными API/архит. практиками

3) Демо должно быть “как в жизни”

То есть:

- ошибки (timeouts, retries)
- idempotency key
- correlation-id
- outbox/event
- sample data
- диаграмма последовательности в README

✅ Итог: да, но в правильном формате

Ты сможешь в Claude собрать проект, который разработчики банка реально смогут использовать, если позиционировать его как:

- **reference architecture**
- **шаблон сервисов**
- **учебный стенд**
- **ADR-доказательство**

А не как “готовый продукт”.

Следующий шаг (чтобы не расплываться)

Я могу прямо сейчас предложить тебе **скелет репозитория** (структура папок, сервисы, таблицы, контракты) под твой кейс:

- `payment-service`
- `decision-hub`
- `limit-module`
- `fraud-module`
- `ledger-service`
- `postgres`
- `docker-compose.yml`

и мини-README “как показать коллегам”.

Ниже — **готовый “идеальный промпт” для Claude Code**, который создаст твой первый проект **Bank Runtime Sandbox** (или Decision Hub): сервисы, БД, контракты, идемпотентность, trace, аудит, сценарии демо (legacy vs v2).
Скопируй целиком и вставь в Claude Code (лучше в режиме “создай/измени файлы в репозитории”).

Ты — senior backend engineer + solution architect в финтехе.
Нужно создать референсный проект для демонстрации банковской архитектуры и runtime-проблем (idempotency, retries, explainability).
Проект должен быть минимальным, но production-like по подходам, чтобы его могли понять и взять за основу реальные разработчики банка.

ЦЕЛЬ ПРОЕКТА
- Показать разницу AS-IS (логика размазана) vs TO-BE (Decision Hub, объяснимость, идемпотентность).
- Поднять локально через docker compose.
- Иметь ясные артефакты: OpenAPI, README, диаграммы (PlantUML), таблицы статусов.
- Не использовать реальные банковские данные, только mock.

ТЕХСТЕК (обязательно)
- Python 3.12 + FastAPI + Uvicorn
- Postgres 16
- SQLAlchemy 2.x (async) + Alembic миграции
- Structured logging (JSON) + correlation-id (X-Correlation-Id), прокидывать через все сервисы
- Dockerfile для каждого сервиса, docker-compose.yml в корне

АРХИТЕКТУРА (4 сервиса + 1 БД)
1) api-gateway (упрощенный) — порт 8000
  - Принимает запросы клиента, добавляет/пробрасывает X-Correlation-Id, валидация базовых полей
  - Проксирует запрос в payment-service
2) payment-service — порт 8001
  - endpoint POST /p2p/transfer
  - хранит transfer в БД со статусами
  - вызывает decision-hub для решения (TO-BE путь)
  - вызывает ledger-mock для “проводки”

- реализует идемпотентность по заголовку Idempotency-Key (обязательно)
- реализует retry-safe поведение: повторный запрос не должен создавать вторую проводку
- дополнительно реализовать endpoint POST /p2p/transfer-legacy, где проверки “if-else” размазаны прямо в payment-service (AS-IS для сравнения)

3) decision-hub — порт 8002
- endpoint POST /decision/evaluate
- возвращает DecisionResult: allowed bool, decision {APPROVE|REJECT|CHALLENGE}, reasons[], rule_version, risk_score(optional)
- правила хранятся как данные в таблице rules (Postgres)
- должны быть: приоритет, активность, версия, условие, действие, reason_code
- поддержать минимум 3 правила:
  - LIMIT_DAILY (сумма+daily_sum > threshold => REJECT)
  - AML_102 (country in blocked list => REJECT)
  - FRAUD_017 (device_trust=LOW and amount> X => CHALLENGE или REJECT)
- должен писать audit trail в таблицу decision_audit: какие правила проверялись, какие сработали, correlation_id, decision_id

4) ledger-mock — порт 8003
- endpoint POST /ledger/posting
- имитирует проводку и иногда “ломается” по параметрам:
  - если header X-Fail-Mode = TIMEOUT => sleep(3)
  - если X-Fail-Mode = ERROR => вернуть 500
- хранит postings в таблице ledger_postings (Postgres) для демонстрации дублей

БАЗА ДАННЫХ (Postgres)
- Одна Postgres для всех сервисов (для простоты демо), но у каждого сервиса свои таблицы/схемы с префиксом.
- Таблицы:
  - payment_transfers (id, client_id, receiver_id, amount, currency, country, status, created_at, correlation_id)
  - payment_idempotency (idempotency_key PK, transfer_id, created_at)
  - decision_rules (rule_id, version, priority, active, condition_json, action, reason_code)
  - decision_audit (id, decision_id, checked_rules[], matched_rules[], result, correlation_id, created_at)
  - ledger_postings (id, transfer_id, status, created_at, correlation_id)

КОНТРАКТЫ И ДОКУМЕНТАЦИЯ (обязательно)
- В корне README.md:
  - Как запустить (docker compose up)
  - 4 сценария демо:
    A) Explainability (почему отказ) — показать reasons[] и audit в БД

    B) Изменение правила без релиза payment-service — поменять decision_rules и повторить запрос
    C) Idempotency — два одинаковых запроса с одним Idempotency-Key => один transfer и один posting
    D) Timeout/Retry — ledger-mock TIMEOUT/ERROR и корректное поведение статусов
  - AS-IS vs TO-BE: чем отличается /transfer-legacy от /transfer
  - Где смотреть логи и correlation-id
- Папка contracts/:
  - OpenAPI yaml для каждого сервиса (или один aggregated) — чтобы dev могли использовать
- Папка docs/:
  - sequence diagrams в PlantUML: legacy и v2
  - таблица статусов payment_transfers.status (NEW, DECIDED, POSTED, FAILED)
- Добавить scripts/:
  - demo.sh который делает curl-запросы по сценариям A–D (с генерацией correlation-id и idempotency-key)

РЕАЛИЗАЦИЯ
- Все сервисы должны стартовать и работать из docker compose.
- Никаких секретов в коде; настройки через env vars.
- В каждом сервисе middleware: correlation-id + JSON logs.
- Обработку ошибок сделать явной: возвращать структурированные ошибки с trace/correlation_id.

СТРУКТУРА РЕПОЗИТОРИЯ (создай так)
```
/
  docker-compose.yml
  README.md
  contracts/
  docs/
  scripts/
  services/
    api-gateway/
    payment-service/
    decision-hub/
    ledger-mock/
  shared/
    logging.py
    correlation.py
```

models.py (если нужно только общие pydantic схемы)

ЗАДАЧА ДЛЯ ТЕБЯ
1) Создай все файлы проекта.
2) Добавь миграции Alembic (или SQL init scripts) чтобы БД поднималась с таблицами и предзаполненными decision_rules.
3) Убедись, что `docker compose up` запускает все сервисы.
4) Добавь в README точные команды curl для сценариев A–D.
5) В demo.sh включи команды, которые показывают:
- legacy вызов
- v2 вызов
- повтор с тем же Idempotency-Key
- изменение rules (через SQL update или через простой admin endpoint в decision-hub)

СРАЗУ НАЧНИ СОЗДАВАТЬ ФАЙЛЫ И КОД.
Не задавай вопросов. Если нужно принять решение — принимай лучшее инженерное решение для демо.

Как использовать (чтобы сработало с первого раза)

1. Создай пустую папку `bank-runtime-sandbox` и открой её в VS Code.
2. Открой Claude Code и вставь промпт выше.
3. После генерации — в терминале:

```
docker compose up --build
```

4. Затем:

```
bash scripts/demo.sh
```

Вот обещанный **короткий “контрольный промпт”** (5–8 строк), который ты вставляешь в Claude Code **после того, как он сгенерирует проект** — чтобы он сам проверил запуск, нашёл типовые косяки и довёл всё до “demo-ready”.

Проверь проект как инженер, который должен завтра показать демо.
1) Прогони мысленно/по коду полный запуск: `docker compose up --build` и найди места, где оно упадёт (env, порты, миграции, зависимости).
2) Исправь всё необходимое, чтобы запуск был стабильным с первого раза, без ручных шагов.
3) Убедись, что БД создаётся, миграции/инициализация применяются автоматически и decision_rules предзаполнены.
4) Проверь, что во ВСЕХ сервисах в логах есть JSON-структура + correlation-id (X-Correlation-Id) и он прокидывается между сервисами.
5) Проверь сценарии A–D из README и scripts/demo.sh: обнови команды/код так, чтобы demo.sh работал полностью и показывал legacy vs v2, idempotency, timeout/error в ledger-mock и explainability (reasons + decision_audit).
Сразу внеси правки в файлы и обнови README (точные команды).

Если хочешь ещё жёстче (для банковского вайба), добавь 2 строки в конец:

6) Добавь единый формат ошибок: {error_code, message, correlation_id} и одинаковые HTTP коды.
7) Добавь минимальные health endpoints /health во всех сервисах и включи их в compose healthcheck.